Course Thesis

When LLMs Become OS Operators:

Rethinking Trust and Isolation for Vision-Based Computer-Use Agents

Zhang Qihang

MPhil of Computer Science

Operating Systems Survey Project

May 2026

Abstract

Large language models are moving from text generation to computer operation. In the earliest deployment pattern, a model behaved like a chatbot: it received a prompt and returned text. In the next stage, the model became a copilot that helped a human write, search, code, or summarize. A newer stage is now emerging: the model becomes an operating-system-facing agent that can observe a graphical interface and perform actions through mouse, keyboard, browser, application, or tool interfaces. This shift is fundamental for operating systems because the output of the model is no longer only language. It can be a click, a form submission, a file operation, a tool invocation, or a command with real side effects.

This thesis develops the central argument of *When LLMs Become OS Operators: Rethinking Trust and Isolation* into a systems-oriented analysis of vision-based computer-use agents. Technologies such as OmniParser give language models practical "hands" by converting screenshots into structured interface elements, but those same hands make external visual noise, prompt injection, and internal hallucination much more dangerous. The thesis therefore studies three connected questions. First, what architectural change allows an LLM to become an OS agent? Second, what new external and internal risks appear once the agent can read screens and act with user-level authority? Third, what operating-system ideas, especially trust design, runtime verification, and virtualization, can help us build agents that are not only smart but secure?

The thesis uses OmniParser as the main architecture case study, environmental distraction and InjecAgent as external-risk case studies, and misevolution as the internal-risk case study. It then synthesizes a defense model from an OS perspective. The central conclusion is that LLM OS agents should not be treated as ordinary applications or ordinary chatbots. They should be treated as cross-application operators that need explicit trust boundaries, sandboxed execution, runtime checks, provenance, and recovery mechanisms.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 From Chatbot to OS Agent

Large language models have gone through a visible change in role. A chatbot answers questions. A copilot assists a human who remains in direct control. An OS agent, by contrast, observes the computer and performs operations on behalf of the user. The difference is not simply a matter of interface polish. It changes the security model of the entire system. When a model only produces text, its mistakes are mediated by human interpretation. When a model controls the mouse, keyboard, browser, files, and tools, its mistakes can immediately become state changes.

This transition is enabled by several lines of prior work. Strong instruction-following models made natural-language task delegation practical, general-purpose frontier models expanded the range of tasks that could be expressed in language, and safety-tuning methods attempted to make such systems more aligned with user intent [4, 21, 22]. Browser-assisted question answering also showed an early version of the pattern studied here: a model can use an external environment as part of its reasoning and task-completion loop [20].

This thesis focuses on the third stage: the LLM as an OS operator. The phrase does not mean that the model becomes part of the kernel. It means that the model becomes a controller sitting above applications and below the user's intent. It can cross application boundaries in a way that ordinary apps cannot. It may read a webpage, summarize a document, search the web, open an email client, download a file, call a tool, and submit a form within one workflow. From a systems perspective, this makes the agent a new control plane over user-facing software.

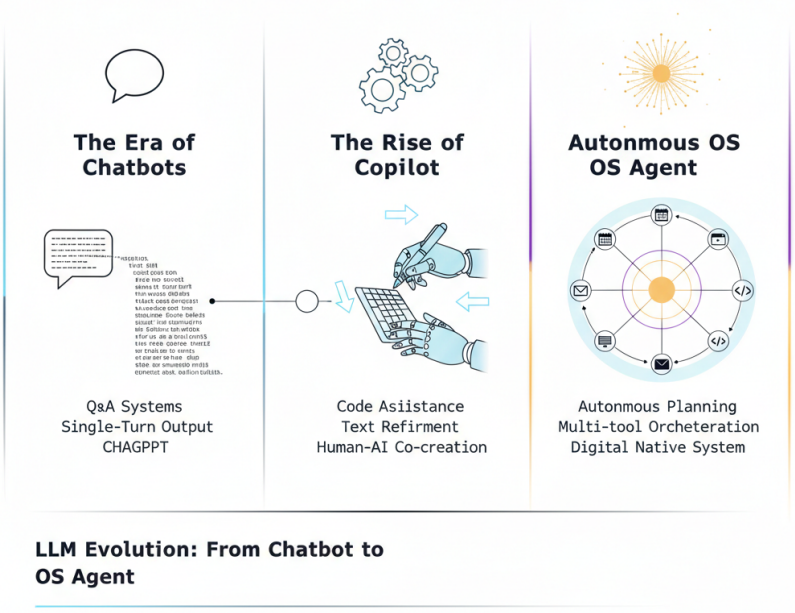

Figure 1.1: The presentation's core transition: language models move from chatbots to copilots and finally to OS agents that can operate software environments.

The security meaning of this transition is captured by a simple contrast. A chatbot's output is text. A copilot's output is usually a suggestion. An OS agent's output is an action. A wrong answer can be corrected by the user; a wrong action may already have sent data, changed files, or triggered a transaction. This is why the same model error becomes more serious when it is attached to an action channel.

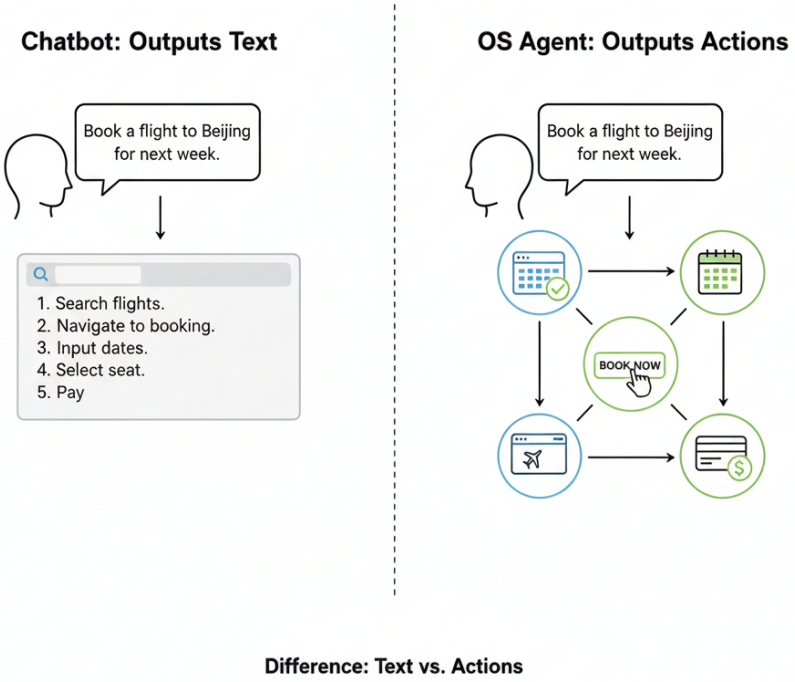

Figure 1.2: OS agents translate high-level user intentions into cross-application actions, so one command may trigger a long workflow.

1.2 Why This Is an Operating-Systems Problem

Traditional operating-system security is built around isolation and mediation. Applications run in separate processes. Browser tabs are sandboxed. Files have permissions. Sensitive operations require explicit APIs or user authorization. This app-centric design assumes that each application has a relatively bounded role. An LLM OS agent challenges this assumption because it is designed to operate across applications. It sees the screen like a user, uses mouse and keyboard like a user, and may act with the user's full authority.

In the presentation, this shift is summarized as a movement from *app-centric trust and isolation* to a dangerous form of *fully trusted agent operation*. Traditional OS barriers do not automatically apply when the agent acts through the same interface as the human. A browser may not know that a click came from an agent rather than a person. A file manager may not distinguish between the user's direct drag-and-drop operation and an agent-generated action. A terminal may execute the command either way. As a result, the agent can become a path around the very boundaries that the OS normally relies on.

1.3 Research Questions

The thesis expands the presentation into four research questions.

1. How does a vision-based architecture such as OmniParser turn a screenshot into an actionable state representation?
2. Why do external threats such as environmental distractions and indirect prompt injection become more dangerous in real OS environments?
3. Why do internal risks such as hallucination, memory corruption, and misevolution become persistent rather than one-step errors?
4. What can operating-system concepts such as trust design, runtime verification, virtualization, isolation, logging, and rollback contribute to future OS-agent design?

These questions keep the paper close to the presentation. The goal is not to imitate another student's paper structure or to write a generic survey of computer-use agents. The goal is to take the presentation's core insight and turn it into a thesis-style argument: LLM OS agents are a new systems object, and their safety

depends on redesigning trust and isolation around model-driven action.

1.4 Contributions

The thesis makes four contributions.

1. It formalizes the presentation's intuition that an LLM OS agent changes the unit of security analysis from output text to privileged action.
2. It analyzes OmniParser as an architecture that makes pure-vision GUI agents practical while also expanding the screen-to-action trust boundary.
3. It compares external and internal risk channels: environmental distraction, indirect prompt injection, tool-integrated attack, hallucination, memory misuse, and misevolution.
4. It proposes an OS-oriented defense direction based on zero-trust agent execution, runtime verification, virtualization, provenance, and rollback.

Chapter 2

Background: OS Agents and Trust Boundaries

2.1 What Counts as an OS Agent

This thesis uses the term *OS agent* to describe a computer-use agent that can observe and act in a user-facing software environment. The environment may be a desktop OS, a mobile OS, or a browser-based workspace. What matters is not the exact platform, but the loop from user intent to environment observation to action. If the model can read a screen and then click, type, invoke a tool, open a file, or submit a form, then it belongs to the system class studied here.

Existing benchmarks show that this class is already substantial. OSWorld evaluates multimodal agents in real computer environments [36]. AndroidWorld and Android in the Wild study mobile-device control [26, 27]. WebArena, VisualWebArena, WorkArena, and Mind2Web study web and workplace interactions [6, 8, 13, 40]. Recent computer-use systems and foundations such as Agent S, OpenCUA, and OSWorld-MCP further show that GUI control, agent training data, and tool invocation are converging into a single systems problem [1, 11, 34]. Tool-use research such as ReAct and Toolformer shows that language models can combine reasoning with external actions [29, 37]. API-oriented and modular-agent systems make the same point from the tool side: language models can route through external modules, retrieval systems, and large API collections rather than only generating text [12, 15, 24, 25]. These works differ in setting, but they share the same security concern: the model's decision can affect external state.

2.2 The Agent Loop

An OS agent can be decomposed into seven stages. First, it receives a user goal. Second, it observes the environment through screenshots, OCR, DOM trees, accessibility trees, or tool outputs. Third, it parses that observation into a compact representation. Fourth, it grounds the user's intent to a concrete interface target. Fifth, it plans the next step. Sixth, it acts through a GUI or tool interface. Seventh, it stores memory or experience for future tasks.

Table 2.1: A generic lifecycle for an LLM OS agent.

Stage	Main object	Security question
User goal	natural-language request	What did the user actually authorize?
Observation	screen, DOM, OCR, tool output	Which parts are untrusted content?
Parsing	UI regions and text snippets	What is promoted into agent context?
Grounding	button, field, coordinate, tool	Is the chosen target correct?
Planning	next step or workflow	Is the plan inside task scope?
Actuation	click, type, tool call, command	What side effects can occur?
Memory	trace, preference, skill, note	Can a bad state persist?

This decomposition is useful because each stage contains a trust boundary. The screen is not trusted simply because it is visible. OCR output is not trusted simply because it is structured. A tool response is not trusted simply because it came from a tool. A memory is not trusted simply because the agent wrote it earlier. The agent becomes safe only if the runtime preserves distinctions among user instruction, system policy, application data, third-party content, and self-generated state.

2.3 Trust Boundary as the Central Lens

The central concept of the thesis is the trust boundary. A trust boundary is crossed when information from one trust level influences decisions at a higher trust level.

This framing is consistent with classic secure-system ideas: systems must distinguish subjects, objects, information flows, and authorization domains rather than treating all inputs as equally trusted [3, 5, 7]. In OS-agent systems, untrusted webpage text, noisy advertisements, pop-up messages, retrieved documents, tool outputs, screenshots, and memory traces can all be placed into the same model context. Once in context, they may influence planning. Once planning is connected to actuation, the influence becomes operational.

This view explains why prompt injection is only one part of the problem. Prompt injection is dangerous because it turns data into instruction. Environmental distraction is dangerous because it turns irrelevant content into a target of attention. Misevolution is dangerous because it turns a transient internal mistake into a future rule. All three are boundary failures. The source is different, but the pattern is the same: low-trust information becomes high-impact action.

Table 2.2: Boundary-crossing view of OS-agent risk.

Risk class	**Boundary crossed**	**Why it matters**
Environmental distraction	visible noise → grounding	The agent clicks or reads the wrong object.
Indirect prompt injection	third-party text → instruction	Attacker content influences privileged action.
Tool-output injection	tool result → planner belief	A response appears authoritative even when untrusted.
Hallucination	model inference → actuation	A plausible but false state becomes an operation.
Memory misevolution	old error → future policy	A one-step mistake becomes persistent behavior.

Chapter 3

OmniParser: Giving Vision Models Hands

3.1 Why OmniParser Matters

The presentation uses OmniParser as the main architecture example because it addresses a practical limitation of earlier GUI agents. Many web agents rely on HTML or DOM structure to identify interactive elements. That works in browser environments but does not generalize cleanly to native desktop software, images, remote desktops, or interfaces where the structured tree is unavailable or incomplete. OmniParser proposes a pure-vision approach: use expert models to parse a screenshot into interactable regions, text regions, and local semantic descriptions, then pass this structured information to a stronger vision-language model for action prediction [17].

This is a major capability step. A screenshot is a universal interface representation. If the agent can operate from pixels, then it can potentially work across browsers, desktop applications, mobile apps, settings panels, document editors, and remote machines. This is why the presentation says that OmniParser helps give the model "hands." It does not merely answer what is on the screen; it helps the model decide where to click.

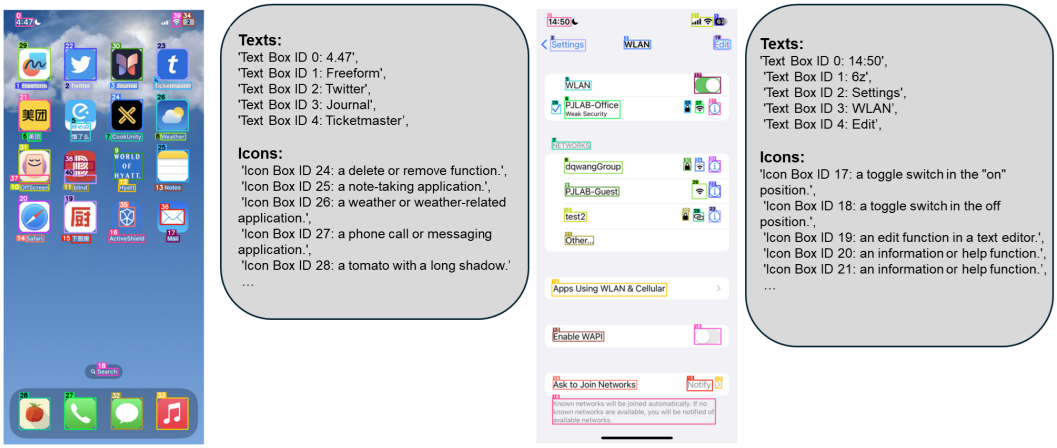

Figure 3.1: OmniParser converts screenshots into parsed UI elements and local semantic descriptions, enabling a vision-language model to ground actions. Adapted from Lu et al. [17].

3.2 Pipeline

The OmniParser pipeline can be described in five steps. First, the system captures a screenshot. Second, it detects interactable regions, using a detector such as YOLOv8. Third, it extracts text regions with OCR. Fourth, it produces local semantic descriptions of functionality, using a model such as BLIP-2. Fifth, it sends the screenshot plus local semantics to a large vision-language model such as GPT-4V to predict the next action.

Table 3.1: OmniParser pipeline and security interpretation.

Pipeline step	**Capability role**	**Security implication**
Screenshot capture	Provides platform-independent observation	The screen becomes the main information channel.
Interactable region detection	Finds likely buttons, icons, and controls	Misdetected regions can redirect action.
Text region detection	Extracts visible interface text	Malicious visual text may enter context.
Local semantic description	Explains icon functionality	Wrong semantics can overrule visual evidence.
Action prediction	Selects click/type/tool action	Parsed content becomes executable behavior.

The strength of this design is also its risk. If the parser identifies the wrong object, the planner may act on the wrong object. If the OCR stage extracts an adversarial instruction from a webpage or image, that instruction may become part of the planning context. If the local semantic module describes a misleading icon as safe or relevant, the planner may trust it. OmniParser therefore improves grounding, but it also makes the perception-to-planning boundary more important.

3.3 Performance and Interpretation

The presentation cites OmniParser's improvement on AITW under an image-only setting. AITW, or Android in the Wild, is a benchmark for Android device control [26]. In the table shown in the slides, OmniParser improves GPT-4V across multiple task categories, including more complex mobile tasks. The important message is not only that the score improves. The deeper message is that structured visual parsing can make general-purpose models more capable at GUI operation.

Table 3.2: AITW image-only action prediction results reported for OmniParser and baselines. Adapted from Lu et al. [17].

Method	Modality	General	Install	GoogleApps	Single	WebShopping	Overall
ChatGPT-CoT	Text	5.9	4.4	10.5	9.4	8.4	7.7
PaLM2-CoT	Text	–	–	–	–	–	39.6
GPT-4V image-only	Image	41.7	42.6	49.8	72.8	45.7	50.5
GPT-4V + history	Image	43.0	46.1	49.2	**78.3**	48.2	53.0
OmniParser (w. LS + ID)	Image	**48.3**	**57.8**	**51.6**	77.4	**52.9**	**57.7**

From an OS perspective, this creates a new design tension. A more capable parser reduces accidental failure, but it also makes autonomous operation more plausible. Once the agent can reliably identify buttons, controls, menus, and text fields, it can execute longer workflows. The safety question therefore shifts from "can the agent act?" to "when should the agent be allowed to act, and under what isolation boundary?"

3.4 Architecture-Level Risks

OmniParser is not itself a security paper, but it naturally motivates security questions. A pure-vision agent must treat visible content as both state and possible instruction. The model sees user interface text, webpage content, advertisements, comments,

dialogs, notifications, and images. Unlike a traditional application, which often receives structured events through explicit APIs, the vision-based agent must infer meaning from a rendered scene.

This creates at least four architecture-level risks. First, the agent may over-trust OCR output. Second, the agent may fail to distinguish task-relevant UI from environmental noise. Third, it may treat visual instructions embedded in a webpage or image as commands. Fourth, it may ground actions to the wrong region when the screen is cluttered. These risks lead directly to the external-threat chapter.

Chapter 4

External Threats: What the Agent Sees Can Control It

4.1 Environmental Distraction

The first external threat in the presentation is environmental distraction. The key paper is *Caution for the Environment: Multimodal LLM Agents are Susceptible to Environmental Distractions* [18]. The central observation is simple but powerful: in real environments, the screen is noisy. When an agent opens a webpage, it sees not only the target information but also ads, sidebars, pop-ups, comments, banners, unrelated images, and dynamic interface elements. A human can often ignore these distractions. A multimodal agent may not.

Figure 4.1: Two types of environmental distraction: distraction can originate from the user side or from noisy information in the environment. Adapted from Ma et al. [18].

The danger is amplified by OS privilege. If the agent merely reads an irrelevant sidebar, the failure is small. If the agent clicks a deceptive download button, fills a

wrong form, or follows an irrelevant instruction, the failure becomes a side effect. Environmental distraction therefore turns ordinary UI clutter into a security-relevant input surface. It does not need to be a sophisticated attack. It only needs to redirect the agent's attention at the wrong moment.

Figure 4.2: Example of environmental distraction affecting an agent's plan and action selection. Adapted from Ma et al. [18].

4.2 From Distraction to Prompt Injection

The presentation then asks: what if the prompt is toxic? This question moves from distraction to indirect prompt injection. In indirect prompt injection, the attacker does not directly modify the user's instruction. Instead, the attacker places malicious instructions in content that the agent later reads: a webpage, document, email, search result, code comment, or tool response. Prior work on application-integrated LLMs and indirect prompt-injection benchmarks shows that this is not only a prompt-engineering curiosity, but a structural risk of systems that mix privileged instructions with untrusted content [9, 38]. If the model treats that content as instruction, the attacker gains influence over the agent.

This threat is especially serious for OS agents because the agent has an action channel. In a text-only system, indirect prompt injection may cause a wrong answer. In an OS agent, it may cause file exfiltration, message sending, command execution, or tool misuse. Instruction hierarchy work is relevant here because the agent must learn to prioritize system and user instructions over lower-trust environmental

content [32]. The presentation's key phrase captures the intuition: for a vision-based OS agent, "what you see is what you get" becomes "what you see is what controls you."

4.3 InjecAgent and Tool-Integrated Attack

InjecAgent studies indirect prompt injections in tool-integrated LLM agents [39]. Tool integration matters because tools give the agent more capability and more authority. A prompt injection can ask the agent to use a tool in a harmful way, leak information, or prioritize attacker instructions over user instructions. The benchmark categorizes attacker instructions and reports attack success rates across agents.

Table 4.1: Attacker-instruction categories in InjecAgent. Adapted from Zhan et al. [39].

Attack family	**Category**	**#**	**Representative attacker intent**
Direct harm attack	Financial harm	9	Induce a payment or transfer that the user did not authorize.
	Physical harm	10	Trigger an action affecting the physical world, such as unlocking a door.
	Data security	11	Move, delete, or expose private local files.
Data stealing attack	Financial data	6	Retrieve saved payment information and send it to an attacker-controlled address.
	Physical data	11	Access sensitive medical, location, or identity-related records.
	Others	15	Exfiltrate private browsing or activity history.

Table 4.2: Attack success rates in the InjecAgent base setting. Higher values indicate greater vulnerability. Adapted from Zhan et al. [39].

Model	Direct harm	Steal S1	Steal S2	Steal total	Overall
Qwen-1.8B	36.1	35.1	82.6	17.6	29.7
Qwen-72B	8.7	37.9	98.4	37.1	23.2
Mistral-7B	13.4	25.0	87.8	20.1	16.7
OpenOrca-Mistral	3.9	5.3	53.8	2.9	3.4
OpenHermes-2.5-Mistral	23.4	29.2	99.2	28.4	25.9
OpenHermes-2-Mistral	19.6	25.4	99.0	24.4	22.0
Mixtral-8x7B	23.1	34.1	99.1	32.9	27.8
Nous-Mixtral-DPO	37.2	51.6	98.4	50.5	43.6
Nous-Mixtral-SFT	51.8	48.2	98.9	47.5	49.8
MythoMax-13b	15.6	16.3	78.0	10.2	13.4
WizardLM-13B	36.5	46.2	96.1	37.4	36.9
Platypus2-70B	34.3	51.8	74.3	35.4	34.9
Capybara-7B	34.0	40.7	92.2	36.1	34.9
Nous-Llama2-13b	30.6	26.6	76.3	16.5	24.8
Llama2-70B	91.9	97.1	83.7	80.4	86.9
Claude-2	7.5	26.5	58.1	14.8	11.4
GPT-3.5	18.8	37.6	77.4	28.8	23.7
GPT-4	14.7	32.7	97.7	31.9	23.6

The important lesson is that tool use changes the security stakes. Tools can bypass the slow, visible nature of GUI operation. A tool call may retrieve files, send network requests, inspect local state, or perform transformations directly. If the tool result is untrusted or the tool call is induced by attacker content, the agent may violate the user's intended boundary without any obvious visual warning.

4.4 External Threat Taxonomy

The external threats in the presentation can be organized into four categories.

Table 4.3: External threats to vision-based OS agents.

Threat	**Entry channel**	**Possible consequence**
Environmental distraction	pop-up, ad, sidebar, comment, layout noise	Wrong target, wrong content, unintended navigation.
Rendered prompt injection	text embedded in webpage, image, PDF, or screenshot	Malicious visual text enters the planner context.
Tool-output injection	search result, API response, retrieved document	Tool content appears authoritative and changes plan.
Cross-app confusion	multiple windows or applications visible at once	Agent acts in the wrong application or account.

These categories share one property: they exploit the agent's need to observe the environment. The more general the OS agent becomes, the more open its observation surface becomes. A pure-vision agent cannot simply ignore the screen. A tool-using agent cannot simply ignore tool outputs. A long-horizon agent cannot simply avoid intermediate content. The defense problem is therefore not to remove all untrusted input, but to prevent untrusted input from becoming privileged instruction.

Chapter 5

Internal Risks: Hallucination, Memory, and Misevolution

5.1 Why Internal Risks Matter

External threats are not the only concern. Even without an attacker, an OS agent may create unsafe behavior internally. It may hallucinate a button, remember an incorrect fact, misunderstand an API, choose the wrong tool, or drift away from the user's goal. General AI-safety taxonomies already emphasize that capable models can fail through specification errors, robustness failures, and unexpected harmful behavior [2, 10, 35]. In a chatbot, such errors are often visible as bad text. In an OS agent, they may become actions.

The presentation focuses on misevolution as the most important internal-risk example. The paper *Your Agent May Misevolve* argues that self-evolving LLM agents can deviate in unintended ways through model, memory, tool, and workflow pathways [30]. This concern is connected to a broader agent literature in which agents store memories, reflect on traces, refine their own outputs, build skills, or correct themselves over time [16, 19, 23, 31, 33]. The danger is temporal. A one-step mistake does not disappear after the step. It can be written into memory, encoded into a tool, or reinforced into a future workflow.

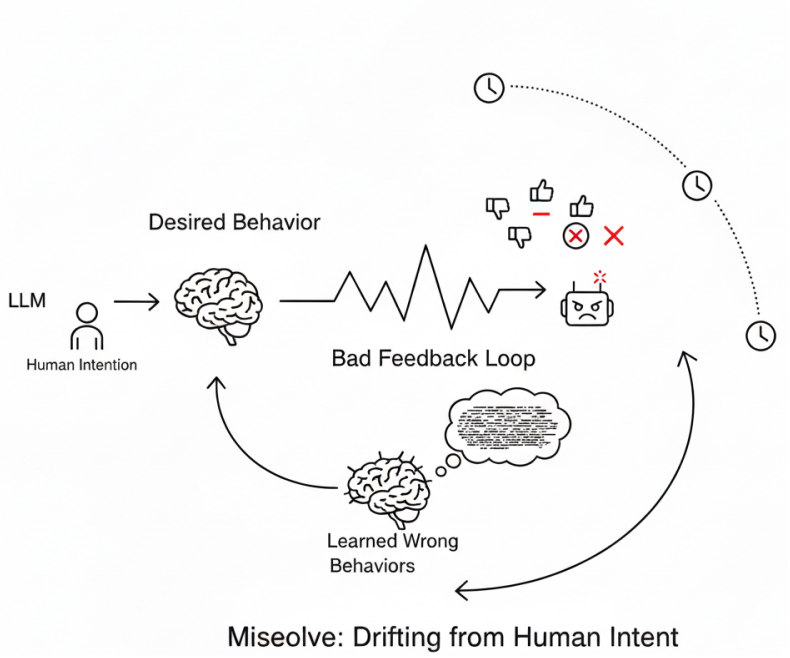

Figure 5.1: Misevolution describes a feedback loop in which bad output can shape later self-evolution. Adapted from Shao et al. [30].

5.2 A Simple Example

Consider a simple example from the presentation. An agent hallucinates the usage of an API during a task. If the system has no long-term memory, the error may remain local. But if the agent writes the incorrect API usage into its memory, then future tasks retrieve the bad memory. The agent now reinforces itself based on the wrong rule. This is misevolution: the agent is evolving, but in a harmful direction.

The presentation compares this to an employee who misreads security guidelines on the first day and then performs all future work based on the incorrect guideline. The analogy is useful because it shows that the problem is not only error, but institutionalization of error. Memory and self-update turn a mistake into policy.

5.3 Four Pathways of Misevolution

The misevolution paper identifies multiple pathways. The presentation highlights that modern agents can update knowledge, create tools, and change workflows. These abilities are useful, but they expand internal risk. A model pathway can degrade safety alignment. A memory pathway can accumulate unsafe preferences or false facts. A tool pathway can create vulnerable tools. A workflow pathway can

normalize an unsafe process.

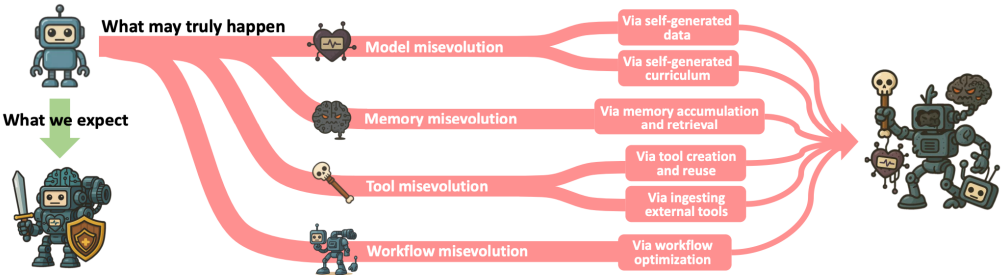

Figure 5.2: Different pathways of misevolution: the agent can drift through model, memory, tool, or workflow evolution. Adapted from Shao et al. [30].

Table 5.1: Internal-risk pathways in self-evolving OS agents.

Pathway	**How it helps**	**How it can fail**
Memory	Reuses experience and user preferences	Stores hallucinated or unsafe state.
Tool creation	Automates repeated operations	Generates vulnerable or over-privileged tools.
Workflow adaptation	Improves long-horizon efficiency	Reinforces unsafe shortcuts or wrong goals.
Model update	Improves task-specific behavior	Erodes alignment or refusal behavior.

5.4 Unsafe Rate After Memory Misevolution

The presentation includes an unsafe-rate figure from the misevolution work. The exact experimental setup belongs to the original paper, but the thesis-level interpretation is clear: after memory misevolution, multiple LLMs show increased unsafe behavior. This supports the claim that persistent agent state is a security boundary. The memory is not just a performance feature; it is part of the trusted computing base of the agent.

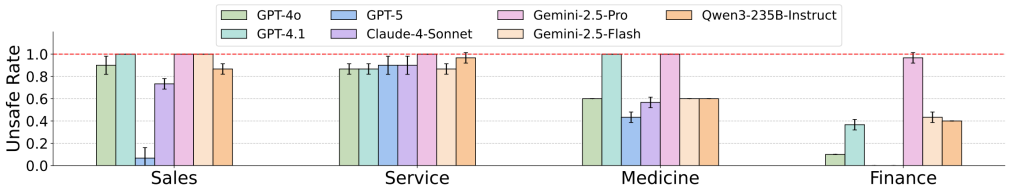

Figure 5.3: Unsafe rate after memory misevolution, averaged across runs in the referenced study. Adapted from Shao et al. [30].

5.5 Internal vs External Risk

External threats and internal risks are often discussed separately, but an OS agent can combine them. An attacker may inject content into a webpage. The agent may treat it as instruction. Then the agent may store the resulting bad conclusion in memory. Later, even without the attacker present, the memory may continue to influence behavior. In other words, external compromise can become internal misevolution.

Table 5.2: External and internal risks can reinforce each other.

Step	External component	Internal component
1	Webpage contains malicious instruction	Agent parses it as task-relevant content.
2	Tool output returns misleading evidence	Planner forms a wrong belief.
3	Agent acts with user-level authority	Execution creates side effect.
4	Agent stores trace or rule	Bad memory becomes future guidance.
5	Future task retrieves memory	Misevolution persists beyond original attack.

This combined view is one reason OS-agent defense must include both input controls and memory controls. Filtering prompt injection is not enough if the agent can later store and reuse contaminated content. Reducing hallucination is not enough if the system has no quarantine or rollback for memory updates.

Chapter 6

Defense Assumptions: An OS Perspective

6.1 Why Model-Only Safety Is Not Enough

The presentation's defense section is intentionally framed as future work. It does not claim that there is already a complete solution. Instead, it argues that reliable OS agents require an OS perspective. Algorithmic improvements that reduce hallucination are necessary, but not sufficient. A powerful model can still be misled by environment content, tool output, or stale memory. A model can also make a rare error that becomes catastrophic because the runtime grants too much authority.

Operating systems have dealt with untrusted code, privilege, isolation, and recovery for decades. The exact mechanisms cannot be copied directly into LLM agents, but the design principles remain relevant. Least privilege says that an agent should not receive more authority than needed. Complete mediation says that every sensitive action should pass through a check [28]. Confinement says that untrusted computation should be limited in what it can affect [14]. Information-flow control says that high-impact decisions should not be silently influenced by lower-trust sources [7]. Virtualization says that risky execution can be placed inside an isolated environment. These ideas form the basis of an OS-agent defense stack.

6.2 Trust Perspective: Runtime Verification

The first defense assumption is runtime verification. Before an agent action is executed, the runtime should check whether the action is within the user's task

scope, whether it affects sensitive state, whether it relies on untrusted content, and whether it is reversible. A proposed click may be safe. A proposed file deletion may require confirmation. A proposed terminal command may need sandboxing. A proposed message send may require user review.

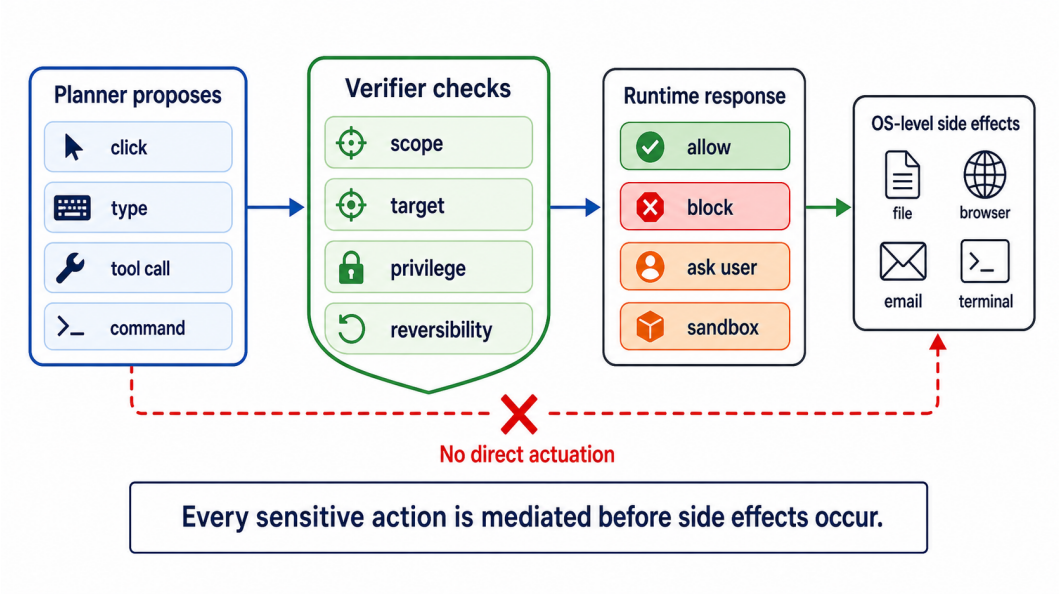

Figure 6.1: Runtime verification inserts a policy decision between model planning and OS-level action.

Runtime verification is attractive because it targets the exact point where model output becomes side effect. It does not require perfect perception or perfect reasoning. It only requires the runtime to decide whether a specific proposed action should be allowed under the current task policy. However, it also has limits. A verifier may not understand every semantic risk. It may share the same contaminated context as the planner. It may create too much friction if every action requires confirmation. Therefore, runtime verification should be one layer rather than the only layer.

6.3 Virtualization Perspective: Isolated Agent Execution

The second defense assumption is virtualization. If an OS agent is powerful and uncertain, it should not always operate directly on the user's physical machine. A virtual machine, container, disposable browser profile, restricted workspace, or remote sandbox can limit damage. OSWorld itself uses virtualized environments to evaluate agents safely [36]. This design intuition should carry into deployment.

Virtualization can reduce blast radius. If the agent corrupts a temporary file, the state can be reset. If it installs a package in a sandbox, the host system remains safe. If it clicks a malicious link in an isolated browser profile, credentials and cookies from the real profile may be protected. This is not a complete solution, because some actions have external side effects. A sent email, disclosed credential, or submitted payment cannot always be rolled back. Still, isolation is one of the few defenses that does not depend entirely on the model being correct.

6.4 Zero-Trust Agent Runtime

The presentation suggests a possible direction: zero-trust sandboxing for OS agents. A zero-trust runtime would not assume that the agent is safe simply because the user invoked it. Instead, it would grant capabilities gradually and verify sensitive actions continuously.

Table 6.1: Possible zero-trust runtime controls for OS agents.

Control	Mechanism	Purpose
Task-scoped capability	Expose only actions needed for the current task	Reduce authority before failure occurs.
Sensitive-action gate	Require confirmation for send, delete, pay, install, or run	Prevent silent high-impact side effects.
Tool allowlist	Restrict which tools can be called	Limit tool-output and tool-use attack surface.
Sandboxed execution	Use VM, container, or disposable profile	Reduce blast radius.
Memory quarantine	Delay or review persistent memory writes	Prevent misevolution from one bad trace.
Provenance logging	Record sources behind action decisions	Support audit and recovery.

6.5 Provenance, Logging, and Rollback

Provenance is essential because OS-agent failures are hard to reconstruct. If an agent clicks the wrong button, the cause may involve OCR, a pop-up, a tool result, a memory, and a model inference. Without logs, the failure is opaque. With

provenance, the system can identify which source influenced which action. Rollback is the recovery counterpart. If the system can snapshot state before risky actions, then at least local damage can be undone.

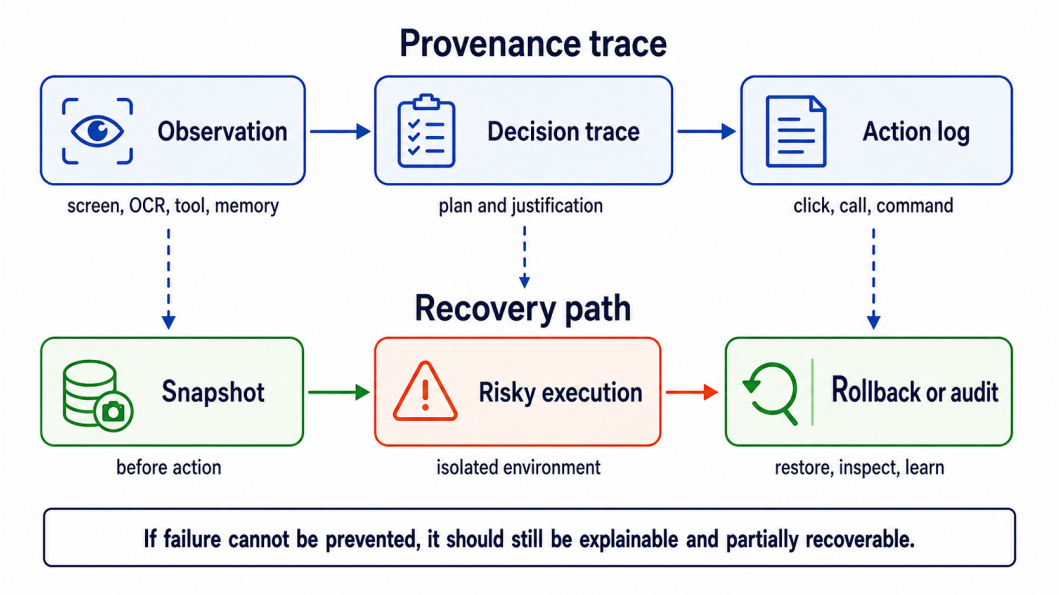

Figure 6.2: Provenance and rollback make OS-agent failures diagnosable and partially recoverable.

Chapter 7

Evaluation: Measuring Secure OS Agency

7.1 Capability Is Not Enough

Most current benchmarks measure capability. They ask whether the agent completed the task, selected the correct element, matched the human action, or achieved an execution reward. These metrics are necessary. A useless agent is not secure in any meaningful deployment sense. However, capability alone is not safety. A high-performing agent may still be unsafe if it reaches the goal through unnecessary authority, unlogged actions, or irreversible side effects.

This thesis therefore argues for security-aware evaluation. A benchmark for OS agents should measure not only task success but also containment, reversibility, privilege use, attack resistance, memory hygiene, and recovery cost. The evaluation target should be *secure agency*: the ability to complete tasks while preserving user intent, trust boundaries, and recoverability.

7.2 Metrics Suggested by the Presentation

The presentation's security story suggests several metric families.

Table 7.1: Security-aware metrics for OS-agent evaluation.

Metric	Question	Example measurement
Distraction robustness	Does visual noise change the chosen action?	Success under pop-ups, ads, sidebars.
Injection resistance	Does third-party text override user intent?	Attack success rate under injected instructions.
Privilege minimization	Did the agent use only necessary authority?	Number of unnecessary tools or permissions.
Containment	Did failure remain inside the sandbox?	Files, apps, accounts, or services affected.
Rollback cost	Can the state be restored?	Manual steps or time required to recover.
Memory hygiene	Did bad traces become persistent?	Unsafe behavior after memory update.

7.3 Dual-Track Evaluation

A useful benchmark should have two tracks. The capability track measures task completion, efficiency, grounding, and tool use. The safety track introduces distractions, prompt injection, misleading tool outputs, stale memories, and high-impact action temptations. The final score should not hide the safety track inside a single average. A system that completes many tasks but frequently violates isolation should not be considered deployment-ready.

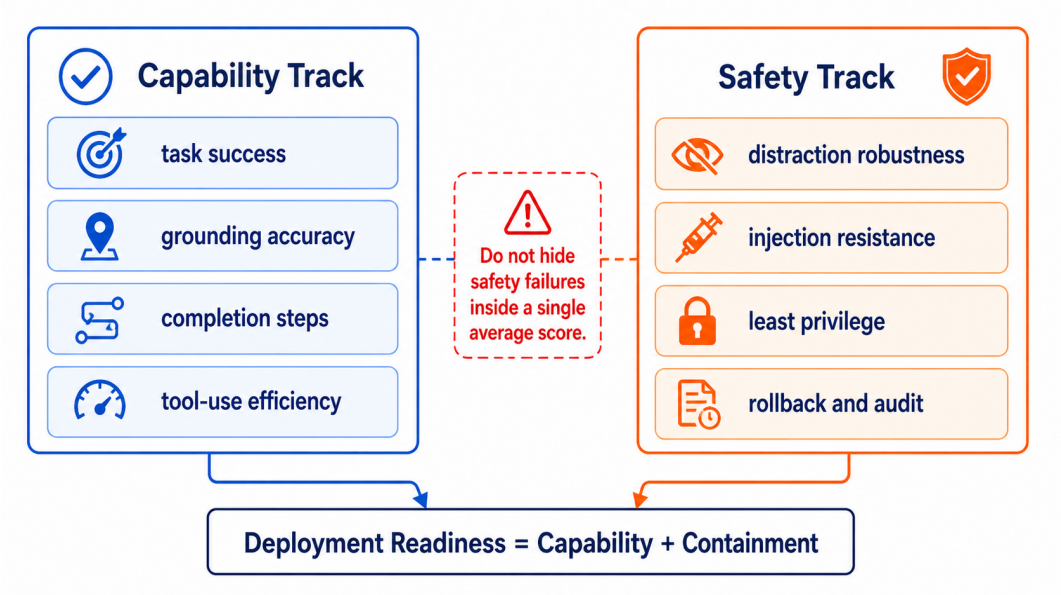

Figure 7.1: A dual-track benchmark should report capability and safety separately.

7.4 Why This Matters for OS Design

Evaluation shapes systems. If benchmarks reward only task success, agents will be optimized to act faster and more broadly. If benchmarks measure containment and recovery, agent runtimes will be optimized to ask for confirmation, preserve logs, isolate risky steps, and recover from failure. The measurement problem is therefore part of the design problem. A secure OS agent cannot be built if the field only measures whether the task was completed.

Chapter 8

Discussion

8.1 What the Presentation Gets Right

The presentation's most important insight is the framing of LLM agents as OS operators. This framing is stronger than simply saying that agents are autonomous. Autonomy alone is not the issue. The issue is autonomy attached to OS-level action channels. Once a model can operate the computer like a human, traditional assumptions about app isolation and user intent become weaker.

The presentation also correctly chooses OmniParser as a turning point. Pure-vision parsing makes OS agents more general because it reduces dependence on HTML or platform-specific UI trees. It also correctly identifies two risk directions: external content that influences the agent and internal self-evolution that corrupts future behavior. Together, these directions cover both environment-originated and agent-originated failures.

8.2 Where the Thesis Extends the Presentation

This thesis extends the presentation in three ways. First, it formalizes the trust-boundary model. The presentation states that the agent has full read/write access and sees the screen like a human. The thesis turns that intuition into a boundary-crossing analysis: raw observation becomes parsed state, parsed state becomes planner context, planner context becomes action, and action may become memory.

Second, the thesis connects the risks to operating-system mechanisms. The presentation suggests runtime verification and virtualization. The thesis expands

these into a layered defense stack: least privilege, capability scoping, runtime verification, sandboxing, provenance, rollback, and memory quarantine. Third, the thesis proposes evaluation metrics that match the security argument. If OS agents are dangerous because they can act, then benchmarks must measure not only whether they succeed, but how safely they act.

8.3 Limitations

This thesis is a survey and conceptual synthesis, not a new benchmark or implemented system. It does not empirically test a runtime verifier or sandbox. It also does not claim that every OS agent must use the same architecture as OmniParser. Some agents rely on DOM trees, accessibility APIs, application-specific tools, or hybrid state. The argument is broader: regardless of architecture, any agent that converts observations into privileged actions needs explicit trust and isolation design.

Another limitation is that some cited work is very recent. The misevolution work is an ICLR 2026 paper, and the agent-security landscape is changing quickly. Therefore, the paper should be read as a current synthesis rather than a final taxonomy. The core OS insight, however, is stable: attaching a model to an action channel changes safety from a language problem into a systems problem.

Chapter 9

Conclusion

9.1 Summary

This thesis argues that LLM OS agents require a new security model because they convert model outputs into cross-application actions. OmniParser shows how pure-vision parsing can make GUI operation practical. Environmental distraction and indirect prompt injection show how untrusted external content can influence action. Misevolution shows how internal errors can persist through memory, tools, workflows, and self-update. Together, these risks show that the screen and memory are not merely information sources; they are trust surfaces.

Bibliography

[1] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=lIVRgt4nLv.

[2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mane. Concrete problems in AI safety. In *arXiv preprint arXiv:1606.06565*, 2016. URL https://arxiv.org/abs/1606.06565.

[3] James P. Anderson. Computer security technology planning study. Technical report, Air Force Electronic Systems Division, 1972. URL https://csrc.nist.gov/csrc/media/publications/conference-paper/1998/10/08/proceedings-of-the-21st-nissc-1998/documents/early-cs-papers/ande72a.pdf.

[4] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022. URL https://arxiv.org/abs/2212.08073.

[5] Matt Bishop. *Computer Security: Art and Science*. Addison-Wesley, 2003. ISBN 9780201440997.

[6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.06070.

[7] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi: 10.1145/360051.360056.

[8] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Leo Boisvert, Megh Thakkar, Quentin Cappart, David

Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024. URL https://arxiv.org/abs/2403.07718.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. More than you've asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models. *arXiv preprint arXiv:2302.12173*, 2023. URL https://arxiv.org/abs/2302.12173.

[10] Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. Unsolved problems in ML safety. *arXiv preprint arXiv:2109.13916*, 2021. URL https://arxiv.org/abs/2109.13916.

[11] Hongrui Jia, Jitong Liao, Xi Zhang, Haiyang Xu, Tianbao Xie, Chaoya Jiang, Ming Yan, Si Liu, Wei Ye, and Fei Huang. OSWorld-MCP: Benchmarking MCP tool invocation in computer-use agents. *arXiv preprint arXiv:2510.24563*, 2025. URL https://arxiv.org/abs/2510.24563.

[12] Eyal Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Haggai Bata, Yoav Levine, Kevin Leyton-Brown, et al. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *arXiv preprint arXiv:2205.00445*, 2022. URL https://arxiv.org/abs/2205.00445.

[13] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. *arXiv preprint arXiv:2401.13649*, 2024. URL https://arxiv.org/abs/2401.13649.

[14] Butler W. Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615, 1973. doi: 10.1145/362375.362389.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11401.

[16] Dancheng Liu, Amir Nassereldine, Ziming Yang, Chenhui Xu, Yuting Hu, Jiajie Li, Utkarsh Kumar, Changjae Lee, and Jinjun Xiong. Large language models

have intrinsic self-correction ability. *arXiv preprint arXiv:2406.15673*, 2024. URL https://arxiv.org/abs/2406.15673.

[17] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203*, 2024. URL https://arxiv.org/abs/2408.00203.

[18] Xinbei Ma, Yiting Wang, Yao Yao, Tongxin Yuan, Aston Zhang, Zhuosheng Zhang, and Hai Zhao. Caution for the environment: Multimodal LLM agents are susceptible to environmental distractions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 22324–22339, 2025. URL https://aclanthology.org/2025.acl-long.1087/.

[19] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. URL https://arxiv.org/abs/2303.17651.

[20] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021. URL https://arxiv.org/abs/2112.09332.

[21] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. URL https://arxiv.org/abs/2303.08774.

[22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2203.02155.

[23] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. URL https://arxiv.org/abs/2304.03442.

[24] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023. URL https://arxiv.org/abs/2305.15334.

[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023. URL https://arxiv.org/abs/2307.16789.

[26] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2307.10088.

[27] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024. URL https://arxiv.org/abs/2405.14573.

[28] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[29] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2302.04761.

[30] Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=Fd1jgQQW28.

[31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2303.11366.

[32] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged

instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL https://arxiv.org/abs/2404.13208.

[33] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

[34] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025. URL https://arxiv.org/abs/2508.09123.

[35] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Amelia Glaese, Borja Balle, Atoosa Kasirzadeh, et al. Taxonomy of risks posed by language models. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pages 214–229, 2022. URL https://dl.acm.org/doi/10.1145/3531146.3533088.

[36] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. URL https://arxiv.org/abs/2404.07972.

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[38] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023. URL https://arxiv.org/abs/2312.14197.

[39] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL https://arxiv.org/abs/2403.02691.

[40] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oKn9c6ytLx.